

FIȘĂ DE LUCRU NR. 3

Informatică - Python | Clasa a IX-a | Curriculum de specialitate

Temă: Subprograme (funcții) în Python

Nume și prenume: _____ Data: _____ Notă: _____

NOȚIUNI TEORETICE

Structura unui subprogram în Python:

```
def nume_functie(param1, param2):    # antet
    # corpul functiei
    instructiune_1
    instructiune_2
    return rezultat                # optional

# Apel:
variabila = nume_functie(valoare1, valoare2)
```

- **def**: cuvânt cheie pentru definirea funcției
- **parametri formali**: variabile locale prin care funcția primește datele de intrare la apel
- **return**: returnează rezultatul și încheie execuția funcției
- variabilele definite în funcție sunt **locale** (nu există în afara ei)

Funcții predefinite utile:

Funcție	Descriere	Exemplu
<code>abs(x)</code>	valoarea absolută a lui x	<code>abs(-5) = 5</code>
<code>round(x, n)</code>	rotunjire la n zecimale	<code>round(3.567, 2) = 3.57</code>
<code>int(x)</code>	conversie la întreg (trunchiere)	<code>int(3.9) = 3</code>
<code>len(lst)</code>	numărul de elemente	<code>len([1,2,3]) = 3</code>
<code>min(lst)</code>	minimul dintr-o colecție	<code>min([3,1,2]) = 1</code>
<code>max(lst)</code>	maximul dintr-o colecție	<code>max([3,1,2]) = 3</code>
<code>sum(lst)</code>	suma elementelor	<code>sum([1,2,3]) = 6</code>

Exemplu rezolvat: Funcția `suma_cifre(n)`

Problemă: Scrieți o funcție care primește un număr natural n și returnează suma cifrelor sale.

Cod Python:

```
def suma_cifre(n):
    suma = 0
    while n > 0:
        suma = suma + n % 10    # adaugam ultima cifra
        n = n // 10            # eliminam ultima cifra
    return suma

# Apel:
numar = int(input("n = "))
print("Suma cifrelor:", suma_cifre(numar))
```

Urmărire execuție pentru suma_cifre(1234):

n (înainte)	cifra = n % 10	suma	n (după n // 10)
1234	4	0 + 4 = 4	123
123	3	4 + 3 = 7	12
12	2	7 + 2 = 9	1
1	1	9 + 1 = 10	0
0	-	n = 0, STOP	return 10

Rezultat: **suma_cifre(1234) = 10** (1 + 2 + 3 + 4 = 10)

EXERCIȚII

Exercițiul 1 (1p) - Completați spațiile libere

Completați funcția care calculează maximul a două numere:

```
def maxim(a, b):
    _____ a > b:
        return _____
    _____:
        return _____

# Apel si afisare:
x, y = 7, 12
print("Maximul este:", _____(x, y))
```

Exercițiul 2 (2p)

Scrieți o funcție `este_prim(n)` care returnează `True` dacă `n` este număr prim și `False` altfel. Folosiți funcția pentru a afișa toate numerele prime mai mici sau egale cu 50.

```
.....

.....

.....
```


.....

Exercițiul 5 (2p)

Urmăriți execuția funcției `este_prim` de mai jos pentru **n = 9**. Completați tabelul cu valorile variabilelor la fiecare pas.

```
def este_prim(n):  
    if n < 2:  
        return False  
    for d in range(2, n):  
        if n % d == 0:  
            return False  
    return True
```

d	n % d	Decizie
2	9 % 2 = 1	nu e 0, continuăm
3	9 % 3 = 0	este 0 -> return False

Concluzie: `este_prim(9)` returnează _____. Explică de ce 9 nu este număr prim: